

Fixed-Point RBF Kernel Evaluation

Range-Reduction LUT + Horner Approximation
ECE 410 — `svm_compute_core` (LUT variant, $\gamma = 0.25$)

Contents

1	Motivation and Problem Statement	1
1.1	Overflow in a Naïve Single-Stage Horner	1
2	Range-Reduction Decomposition	2
2.1	Computing P , I , and F in Q6.10	2
3	Integer Part: EXP_INT_LUT	3
4	Fractional Part: 15th-Order Horner Polynomial	3
4.1	Q6.10 Coefficient Quantisation	3
4.2	Hardware Horner Recurrence	4
5	Combined Kernel Output	4
6	Pipeline Latency	4
7	Error Analysis	5
8	Summary of Notation	5

1 Motivation and Problem Statement

The RBF (Gaussian) kernel between a feature vector $\mathbf{x} \in \mathbb{R}^{256}$ and a support vector $\mathbf{s} \in \mathbb{R}^{256}$ is

$$K(\mathbf{x}, \mathbf{s}) = \exp(-\gamma \|\mathbf{x} - \mathbf{s}\|^2), \quad \gamma = 0.25. \quad (1)$$

All operands are stored in **Q6.10** fixed-point format: a 16-bit signed integer whose value represents a real number scaled by $2^{10} = 1024$. The squared Euclidean distance $D = \|\mathbf{x} - \mathbf{s}\|^2$ can reach approximately 60 in floating-point units, so the exponent argument $\gamma D \leq 0.25 \times 60 = 15$ spans the range $[0, 15]$.

1.1 Overflow in a Naïve Single-Stage Horner

A naïve approach evaluates $e^{-\gamma D}$ directly by applying the Horner polynomial for e^z at $z = -\gamma D \in (-15, 0]$. In Q6.10 the intermediate product $\gamma_{\text{int}} \times D_{\text{int}}$ can reach $256 \times 64,000 \approx 1.6 \times 10^7$, which easily overflows a 16-bit accumulator and wraps to a large positive value. The Horner evaluator then returns ≈ 1.0 instead of $e^{-15} \approx 3 \times 10^{-7}$, collapsing classifier accuracy to near-random ($\approx 20\%$).

2 Range-Reduction Decomposition

The fix exploits the multiplicative identity

$$e^{-P} = e^{-I} \cdot e^{-F}, \quad P = I + F, \quad (2)$$

where $I = \lfloor P \rfloor \in \mathbb{Z}_{\geq 0}$ is the integer part and $F = P - I \in [0, 1)$ is the fractional part. Only e^{-F} is approximated by the Horner polynomial; its argument is always confined to $(-1, 0]$, which is well inside the polynomial's accurate range. The factor e^{-I} is retrieved from a small read-only LUT.

Remark (field-programmable γ). Because the decomposition $e^{-P} = e^{-I} \cdot e^{-F}$ holds for any value of P , and because I and F are simply the integer and fractional parts of $P = \gamma D$ regardless of how γ was obtained, **neither the LUT entries nor the Horner coefficients depend on γ** . The LUT stores e^{-i} for the non-negative integers $i = 0, 1, \dots, 15$ — a universal constant table. The Horner polynomial approximates e^x for $x \in (-1, 0]$ — also independent of γ . Reprogramming γ via the `param_write_en` interface at runtime only changes the product $P_{\text{int}} = \gamma_{\text{int}} \times D_{\text{int}}$ computed in the **SCALE** stage; all downstream logic (LUT lookup, Horner recurrence, final multiply) is unaffected. The one practical limit is saturation: for $I \geq 8$ the LUT already returns zero (since $e^{-8} \cdot 1024 \approx 0.34$ rounds to zero in Q6.10), so any γ large enough to push $I \geq 8$ for a given distance will yield $K_{\text{out}} = 0$, which is the mathematically correct saturated result.

Design note (recommended extension). A silent all-zero kernel matrix is indistinguishable from a valid low-similarity result, so the MCU has no way of knowing the classifier has silently degraded. A lightweight on-chip guard can prevent this. Define a programmable saturation threshold γ_{max} such that

$$\gamma_{\text{int}} > \gamma_{\text{max, int}} \implies \text{error} \leftarrow 1 \quad (3)$$

latched in the **IDLE** $\rightarrow$ **LOAD_FIFO** transition alongside the existing SV-count validation. A reasonable default is $\gamma_{\text{max}} = 8.0$ ($\gamma_{\text{max, int}} = 8192$ in Q6.10): at this value even the minimum non-zero distance $D = 2^{-10}$ (one Q6.10 LSB) gives $I = \lfloor 8.0 \times 2^{-10} \rfloor = 0$, so no legitimate distance is silently zeroed. Alternatively, a runtime check can count how many kernel outputs in a batch equal zero and assert **error** if the fraction exceeds a configurable threshold (e.g. $> 50\%$ zero), which catches mis-programmed γ without requiring a hard static limit.

2.1 Computing P , I , and F in Q6.10

Let γ_{int} and D_{int} denote the Q6.10 integer representations of γ and D respectively (i.e. $\gamma_{\text{int}} = \lfloor \gamma \cdot 1024 \rfloor$, $D_{\text{int}} = \lfloor D \cdot 1024 \rfloor$). The hardware computes:

$$P_{\text{int}} = \gamma_{\text{int}} \times D_{\text{int}} = \lfloor \gamma D \cdot 2^{20} \rfloor \quad (36\text{-bit unsigned product}), \quad (4)$$

$$I = P_{\text{int}} \gg 20 = \lfloor \gamma D \rfloor, \quad (5)$$

$$F_{\text{q}} = (P_{\text{int}} \gg 10) \ \& \ 0\text{x}3\text{FF} = \lfloor F \cdot 1024 \rfloor \in [0, 1023]. \quad (6)$$

The Horner input is then set to

$$x = -F_{\text{q}} \in [-1023, 0], \quad (7)$$

which represents the real value $x_{\text{real}} = -F \in (-1, 0]$ in Q6.10. If $I \geq 8$ (i.e. $\gamma D \geq 8$, meaning $e^{-I} < 2^{-11} < 1 \text{ LSB}$), the LUT returns zero and the entire kernel output is set to zero without computing the Horner polynomial.

3 Integer Part: EXP_INT_LUT

The 16-entry read-only LUT stores precomputed values

$$\text{EXP_INT_LUT}[i] = \text{round}(e^{-i} \cdot 1024), \quad i = 0, 1, \dots, 15. \quad (8)$$

Table 1: EXP_INT_LUT contents (16-bit Q6.10, 32 bytes total).

i	e^{-i} (float)	LUT value (integer, Q6.10)
0	1.000000	1024
1	0.367879	377
2	0.135335	139
3	0.049787	51
4	0.018316	19
5	0.006738	7
6	0.002479	3
7	0.000912	1
8–15	$< 3.4 \times 10^{-4}$	0

The LUT value at the selected index is latched into the 16-bit register `lut_val` during the SCALE2 pipeline stage.

4 Fractional Part: 15th-Order Horner Polynomial

The Taylor series for e^x around $x = 0$ is

$$e^x = \sum_{n=0}^N \frac{x^n}{n!} + O(x^{N+1}). \quad (9)$$

For $x = -F \in (-1, 0]$ and $N = 15$ the truncation error is $|x|^{16}/16! < 10^{-13}$, far below the Q6.10 resolution of 2^{-10} . The polynomial is evaluated in **Horner form** to minimise multiply-add operations:

$$e^x \approx c_0 + x(c_1 + x(c_2 + x(\dots + x(c_{14} + x \cdot c_{15}) \dots))), \quad c_n = \frac{1}{n!}. \quad (10)$$

4.1 Q6.10 Coefficient Quantisation

Each coefficient $c_n = 1/n!$ is stored as the 16-bit integer $\hat{c}_n = \lfloor c_n \cdot 1024 \rfloor$. Coefficients smaller than $1/1024 \approx 9.77 \times 10^{-4}$ are rounded to zero; the table below lists the stored values used in the RTL.

The dominant quantisation error occurs at $n = 6$: the exact value $1/720 \approx 0.001389$ is stored as $1/1024 \approx 0.000977$, introducing an error of $\approx 4.1 \times 10^{-4}$. For $|x| \leq 1$ the propagated polynomial error stays well below the experimentally measured peak of 5.5×10^{-4} .

Table 2: Horner coefficients in Q6.10 (COEFF_00–COEFF_15 in `svm_compute_core.sv`).

n	$c_n = 1/n!$ (float)	$\hat{c}_n$ (integer)	RTL constant
0	1.000000	1024	COEFF_00
1	1.000000	1024	COEFF_01
2	0.500000	512	COEFF_02
3	0.166667	170	COEFF_03
4	0.041667	42	COEFF_04
5	0.008333	8	COEFF_05
6	0.001389	1	COEFF_06
7–12	$< 9.77 \times 10^{-4}$	0	COEFF_07--12
13–15	≈ 0	1	COEFF_13--15*

*Set to 1 rather than 0 to prevent the multiply chain from dead-ending on a zero factor.

4.2 Hardware Horner Recurrence

Each Horner step computes one multiply-accumulate in a pipelined fashion. Letting r_k denote the running result after processing coefficient c_k from the inside out:

$$r_{15} = \hat{c}_{15}, \quad (11)$$

$$r_k = \hat{c}_k + \left\lfloor \frac{x \cdot r_{k+1}}{1024} \right\rfloor, \quad k = 14, 13, \dots, 0. \quad (12)$$

The division by 1024 (arithmetic right-shift by 10) is the Q6.10 renormalisation after each 16-bit $\times$ 16-bit $\rightarrow$ 32-bit multiply:

$$\text{temp_h} = x_{\text{int}} \times r_{k+1, \text{int}}, \quad \left\lfloor \frac{x \cdot r_{k+1}}{1024} \right\rfloor = \text{temp_h}[25:10]. \quad (13)$$

After the final step the result $r_0 \approx e^{-F} \cdot 1024$ is clamped to $[0, 1024]$ to handle rounding artefacts:

$$\text{horner_clamp} = \text{clamp}(r_0, 0, 1024). \quad (14)$$

5 Combined Kernel Output

The two factors are multiplied and renormalised:

$$K_{\text{out}} = \left\lfloor \frac{\text{lut_val} \times \text{horner_clamp}}{1024} \right\rfloor = \left\lfloor e^{-I} \cdot 1024 \cdot e^{-F} \cdot 1024 \cdot 2^{-10} \right\rfloor \approx e^{-\gamma D} \cdot 1024 \quad (15)$$

in Q6.10. The maximum product is $1024 \times 1024 = 2^{20}$, which fits in a 32-bit unsigned register; bits $[25:10]$ of the product deliver the Q6.10 result. The output is never clamped at this stage because both factors are already bounded to $[0, 1024]$.

6 Pipeline Latency

The `horner_engine` module executes the following sequential states, one per clock cycle at 50 MHz:

Table 3: Pipeline stages and their operations (18 cycles total).

Cycle	State	Operation
1	SCALE	$P_{\text{int}} \leftarrow \gamma_{\text{int}} \times D_{\text{int}}$ (36-bit product)
2	SCALE2	Extract I , look up <code>lut_val</code> ; set $x \leftarrow -F_q$
3	HORNER_14	$r \leftarrow c_{15}$; $\text{temp_h} \leftarrow x \cdot c_{15}$
4	HORNER_13	$r \leftarrow c_{14} + \text{temp_h}[25:10]$
$\vdots$	$\vdots$	$\vdots$
17	HORNER_0	$r \leftarrow c_1 + \text{temp_h}[25:10]$
18	OUTPUT	$r_0 \leftarrow c_0 + \text{temp_h}[25:10]$; clamp; multiply LUT; emit <code>kernel_out</code>

7 Error Analysis

Define the absolute error as $\varepsilon = |K_{\text{out}}/1024 - e^{-\gamma D}|$. Three sources contribute:

1. **LUT quantisation.** The integer-part factor is rounded to the nearest integer in Q6.10. Relative error $\leq 0.5/(e^{-i} \cdot 1024)$; largest for $i = 7$ (stored as 1, exact ≈ 0.934 , relative error $\approx 7\%$). Absolute contribution $\leq 7 \times 10^{-4}$ at the kernel output.
2. **Coefficient quantisation.** The dominant term is c_6 : stored as $1/1024$, exact $1/720$, error $\approx 4.1 \times 10^{-4}$ per unit of $|x|^6$. For $|x| \leq 1$ the polynomial error $\leq 4.1 \times 10^{-4}$.
3. **Multiply-shift truncation.** Each of the 15 Horner shifts truncates a half-LSB on average, accumulating $\leq 15 \times 2^{-11} \approx 7 \times 10^{-3}$ in the worst case; however cancellation across terms keeps the net effect below 2×10^{-4} .

The empirically measured maximum over $D^2 \in [0, 60]$ is

$$\max_{D^2 \in [0, 60]} \varepsilon = 5.5 \times 10^{-4}, \quad (16)$$

which is within the Q6.10 LSB width of $1/1024 \approx 9.77 \times 10^{-4}$.

8 Diagnostic Error Codes

The `svm_compute_core` module exposes two error outputs to the host MCU:

- **error [1-bit]** — asserts high on the first fault and *holds* until `rst_n` is de-asserted. Backward-compatible with the single-bit flag used in prior RTL versions.
- **error_code [3:0]** — latches the code of the *first* fault detected and holds it until reset. A value of `4'h0` means no fault has occurred.

Codes are assigned by a combinational priority encoder; the highest-priority active condition wins.

Implementation notes

- **Sticky latch.** Once `error_code` $\neq 0x0$, it does not update even if a second fault of lower priority occurs. This preserves the *root cause* for the MCU to read after `done` or after a watchdog timeout.

Table 4: Error code definitions (`error_code[3:0]`, priority high $\rightarrow$ low).

Code	Constant	Priority	Condition and MCU action
0x0	ERR_NONE	—	No fault. Normal operation.
0x1	ERR_SV_ZERO	High	$\sum \text{sv_count} = 0$ when the FSM leaves <code>IDLE</code> . No support vectors were programmed before <code>start</code> . <i>Action: re-program <code>num_sv_per_class</code> and pulse <code>rst_n</code>.</i>
0x2	ERR_SV_OVERFLOW	$\uparrow$	$\sum \text{sv_count} > \text{NUM_SV}$ (hardware capacity exceeded). <i>Action: reduce per-class SV counts and reset.</i>
0x3	ERR_ILLEGAL_STATE	$\uparrow$	FSM reached the <code>default</code> (undefined) branch. Indicates a synthesis/simulation mismatch or glitch. <i>Action: assert <code>rst_n</code>; inspect waveform.</i>
0x4	ERR_GAMMA_SAT	$\downarrow$	$\gamma_{\text{int}} > 8192$ (i.e. $\gamma > 8.0$ in Q6.10). All kernel outputs will be zero because $e^{-I} = 0$ for $I \geq 8$ in the LUT. See Section 3. <i>Action: re-program <code>gamma_reg</code> to a value $\leq 0x2000$ and reset.</i>
0x5	ERR_FIFO_OVERFLOW	Low	The input FIFO was full while <code>qspi_valid</code> was asserted during <code>LOAD_FIFO</code> . One or more feature words were silently dropped, corrupting the feature vector. <i>Action: reduce QSPI clock rate or increase <code>FIFO_DEPTH</code>; reset.</i>

- **Reset to clear.** The only way to clear `error` and `error_code` is to assert `rst_n` low for at least four clock cycles. A bare `start` pulse does not clear them.
- **FIFO overflow detection.** The `ERR_FIFO_OVERFLOW` flag is a separate sticky register inside the core that asserts whenever `fifo_full` && `qspi_valid` during `LOAD_FIFO`. It feeds into the priority encoder on every cycle, so the error is reported even if the FSM subsequently completes normally.
- **Future codes** 0x6–0xF are reserved for extension (e.g. kernel-output watchdog, work-RAM address overflow).

9 Summary of Notation

Table 5: Symbol definitions.

Symbol	Definition
γ	RBF bandwidth; $\gamma = 0.25$, stored as $\gamma_{\text{int}} = 256$ (Q6.10)
D	Squared Euclidean distance $\ \mathbf{x} - \mathbf{s}\ ^2$, 20-bit Q6.10
P	Exponent argument γD ; $P_{\text{int}} = \gamma_{\text{int}} \times D_{\text{int}}$ (36-bit)
I	Integer part of P ; $I = P_{\text{int}} \gg 20 \in \{0, \dots, 15\}$
F	Fractional part of P ; $F \in [0, 1)$
F_q	Q6.10 representation of F ; $F_q = (P_{\text{int}} \gg 10) \& 0\mathbf{x3FF}$
x	Horner argument; $x = -F_q \in [-1023, 0]$ (integer), $-F \in (-1, 0]$ (real)
$\hat{c}_n$	Q6.10 coefficient $\lfloor n!^{-1} \cdot 1024 \rfloor$
<code>lut_val</code>	$\text{round}(e^{-I} \cdot 1024)$, 16-bit
<code>horner_clamp</code>	$\text{clamp}(r_0, 0, 1024) \approx e^{-F} \cdot 1024$, 16-bit
K_{out}	Final kernel output; $\lfloor \text{lut_val} \times \text{horner_clamp} / 1024 \rfloor$, 16-bit Q6.10